

SELF-VLA: A Skill Enhanced Agentic Vision-Language-Action Framework for Contact-Rich Disassembly

Abstract—Disassembly automation has long been pursued to address the growing demand for efficient and proper recovery of valuable components from end-of-life (EoL) electronic products. Existing approaches have demonstrated promising and regimented performance by decomposing the disassembly process into different subtasks. However, each subtask typically requires extensive data preparation, model training, and system management. Moreover, these approaches are often task- and component-specific, making them poorly suited to handle the variability and uncertainty of EoL products and limiting their generalization capabilities. All these factors restrict the practical deployment of current robotic disassembly systems and leave them highly reliant on human labor. With the recent development of foundation models in robotics, vision-language-action (VLA) models have shown impressive performance on standard robotic manipulation tasks, but their applicability to complex, contact-rich, and long-horizon industrial practices like disassembly, which requires sequential and precise manipulation, remains limited. To address this challenge, we propose SELF-VLA, an agentic framework that combines role-based VLA modules with explicit disassembly skills. SELF-VLA assigns approach and pickup recovery to VLA policies, while programmed skills execute contact-rich disassembly and placement. A stop-token-based handoff mechanism coordinates control transfer between these stages. Across seven component disassembly tasks, SELF-VLA improves full-task success rates by an average of 57.9 percentage points over the corresponding end-to-end policies using OpenVLA-OFT and $\pi_{0.5}$. Video results are available at <https://self-vla.github.io/>

I. INTRODUCTION

Electronic waste (e-waste) is one of the fastest-growing waste streams, with only 22.3% of 62 million tonnes globally in 2022 documented as recycled. Besides the environmental issues associated with improper treatment, end-of-life (EoL) electronics contain substantial quantities of rare earth elements (REEs), whose loss further aggravates existing supply chain risks [1]. Disassembly is an important operational step in recovering components and materials for recycling and remanufacturing. However, collected EoL products (e.g., desktops, phones, and laptops) exhibit significant variability across different brands and considerable uncertainty due to unpredictable usage conditions, requiring complex decision-making and planning during disassembly operations [2]. Traditionally, manual disassembly has dominated this practice because humans can effectively handle the uncertainty and variability inherent in different EoL products. However, the high labor costs and low efficiency of manual processes cannot meet the increasing demand for EoL desktop disassembly and render it unprofitable [3]. Additionally, health concerns during the disassembly operation further motivate

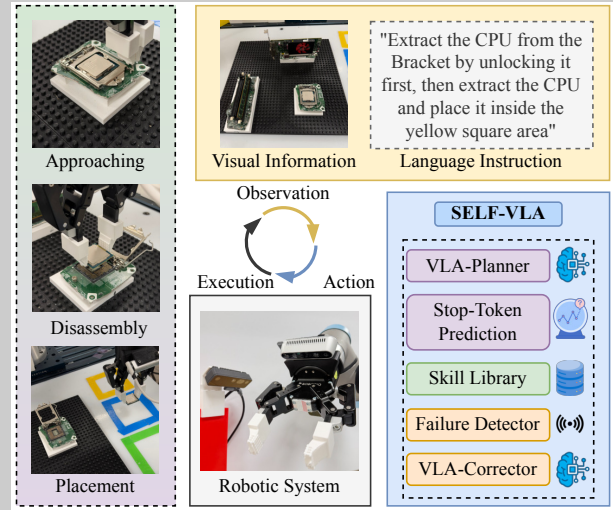


Fig. 1. SELF-VLA framework for contact-rich disassembly. Visual observations and language instructions guide SELF-VLA, which activates the appropriate module and sends actions to the robotic system for execution. Stop-token prediction controls the handoff from the VLA-Planner to a skill. The failure detector identifies pickup failures during skill execution and activates the VLA-Corrector.

the integration of robots to reduce human involvement [4]. Many current robotic-assisted disassembly approaches rely on a regimented sequential pipeline consisting of perception [5], task planning [6], [7], motion planning [8], and manipulation [9]. While effective in a controlled setting, these task-specific designs rely heavily on explicit modeling and staged engineering, often requiring stage-specific data preparation, models or rules, and integration across modules.

Large foundation models have advanced the deployment of artificial intelligence (AI) across different domains [10]. Their pre-training on large-scale multi-modal datasets enables strong generalization. In robotics, vision-language-action (VLA) models, which leverage vision-language foundation models as reasoning backbones to directly generate executable robot actions, have emerged as a promising direction for both industry and academia [11]. VLA models aim to learn from large-scale human demonstrations, enabling robots to interpret perceptual inputs with language instructions and execute tasks directly without multi-stage splits. While current VLA models have shown strong results for robotic tabletop daily tasks [12]–[14], their applicability to more complex long-horizon and contact-rich settings remains limited. Training datasets such as LIBERO [15] and

DROID [16] primarily consist of relatively simple daily-life manipulation tasks with loose constraints and lack industry-level operational data. In contrast, industrial environments demand high-precision manipulation in sequential, contact-rich operations. Beyond basic pick-and-place tasks, these scenarios often involve standardized extraction procedures that must adhere to strict operational constraints. Meeting such requirements remains challenging for current VLA models, even after domain-specific fine-tuning [17], [18]. In these end-to-end approaches, a single continuous policy must accommodate different stages that impose distinct execution constraints. Recent work such as PAPO-VLA distinguishes planning and execution as functional roles within a single VLA policy [19]. This raises the question of how to retain the flexibility of end-to-end learning while incorporating stage-specific execution constraints.

Recently, agentic frameworks built upon large language models (LLMs) have attracted increasing attention. These frameworks typically operate through an iterative loop involving task understanding, planning, tool-augmented execution, and feedback-driven evaluation [20]. Such frameworks extend LLM applications from response generation to multi-step task execution. The recent emergence of LLM agent skills further supports this process by formalizing procedural knowledge into structured and reusable workflows that explicitly guide both reasoning and execution [21]. This structured agentic paradigm motivates extending the applicability of VLA models in robotics. In industrial disassembly, precise and consistent contact-rich manipulation remains challenging for current VLA models. However, these operations often follow component-specific standard procedures. Reaching a suitable starting pose requires adaptation to the observed component, while these procedures specify the subsequent removal sequence. Sorting then involves placement at task-defined destinations, with limited need for continued VLA adaptation. These characteristics motivate an agentic approach that coordinates VLA control with reusable skills according to the execution requirements of each task stage.

Motivated by this agentic perspective, we propose SELF-VLA, a VLA framework with explicit skill integration for robotic disassembly tasks, as shown in Fig. 1. The framework consists of three coordinated components: a VLA-planner, a skill library, and a VLA-corrector. The VLA-planner interprets language instructions, visual observations, and robot state to generate continuous control actions toward the target component. The planner and corrector learn to signal termination through a stop token encoded as a reserved value in the gripper output channel. A stop-token prediction (STP) module determines planner-to-skill handoff using current action predictions and recent prediction and measured-state histories. The framework then invokes the corresponding skill from the skill library. The skill executes the manipulation along a predefined waypoint sequence with failure detection after pickup or during transport. If a failure is detected, the VLA-corrector is activated to regrasp the component, ending correction when it predicts the stop token. After successful regrasping and handoff, the skill resumes

only the placement phase at the given destination. The main contributions of this work can be summarized as follows:

- We propose SELF-VLA, a skill-enhanced agentic VLA framework that integrates structured disassembly skills with failure recovery for long-horizon contact-rich manipulation.
- We introduce a stop-token-based handoff mechanism that pairs history-aware stop-token prediction for planner-to-skill transfer with corrector-predicted STOP tokens for terminating correction and enabling skill resumption.

The remainder of the paper is organized as follows. Section 2 reviews related work on robotic disassembly and VLA models. Section 3 presents the SELF-VLA framework and the construction of its training dataset. Section 4 describes the experimental setup, evaluations, and detailed result analysis. Section 5 concludes the paper.

II. RELATED WORK

A. Robotic Disassembly

Current robotic-assisted disassembly approaches rely on a sequential pipeline that processes multi-modal information (e.g., visual information, CAD data, product information, and environmental information) to plan robot actions at different stages [2]. Prioli et al. [22] extract interference matrices from CAD files to build precedence matrices, providing collision-free robotic disassembly sequence planning. Zhang et al. [23] propose a YOLOv4-based framework to detect and localize screws from visual inputs and recommend matched tools for robotic unscrewing tasks. However, such information is usually incomplete or unavailable in real-world scenarios, limiting the practical feasibility of these algorithms [24]. Liu et al. [3] propose an automated selective disassembly system for EoL phones that achieves high throughput under structured operating conditions. However, it relies on specific phone structures and has limited generalization capability.

Human-robot collaborative disassembly (HRCd) has been proposed as an intermediate solution that integrates humans' dexterous manipulation and flexible decision-making with robots' reliability, repeatability, and precision [7], [25]. Lee et al. [7] propose a resource-constrained optimization algorithm to perform task planning for HRCd, reducing the operation time subject to safety constraints. However, human involvement adds uncertainty to robot motion planning, particularly when predicting human actions for safe operation. Tian et al. [26] distill a diffusion-based motion predictor into a lightweight one-step MLP model for real-time stochastic 3D human motion prediction in HRCd. However, robot behavior in shared workspaces may affect operator trust and reduce task efficiency [27]. Achieving reliable robotic automated disassembly without human intervention across diverse EoL products remains a long-term objective [24].

B. Vision-Language-Action Models

Vision-Language-Action (VLA) models directly map visual observations and language instructions to robot actions. These models differ in how robot actions are represented

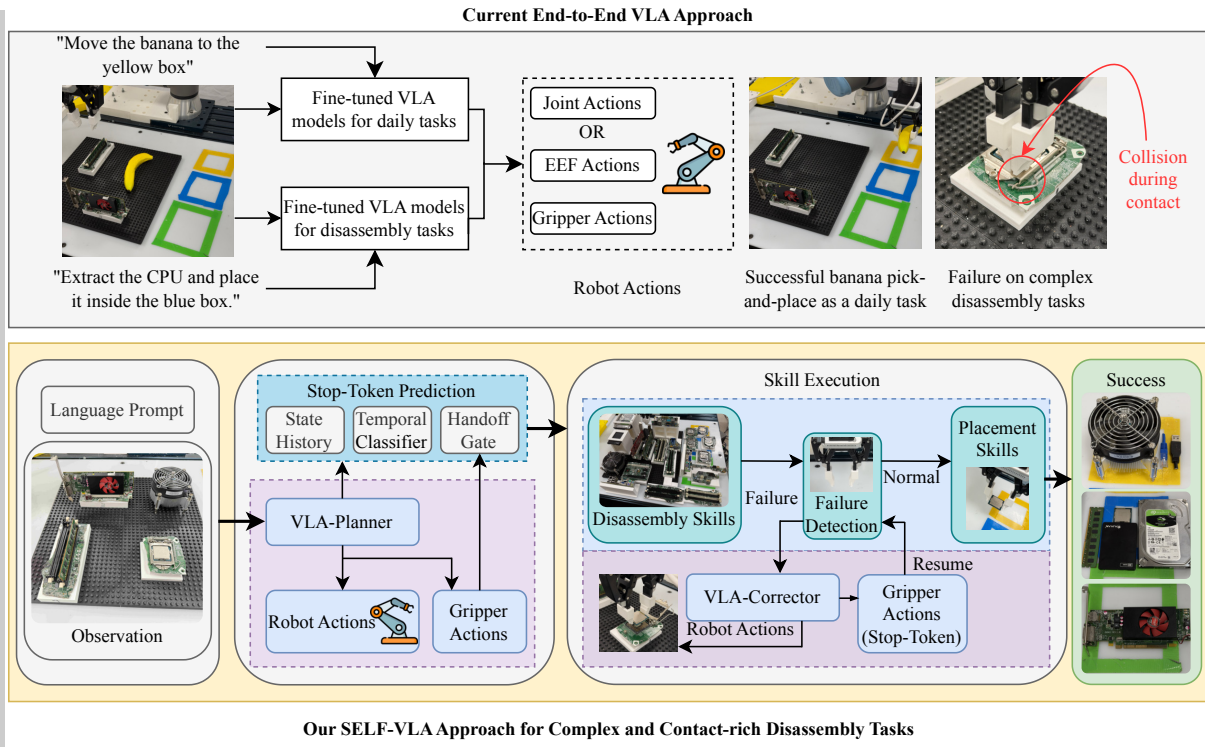


Fig. 2. Comparison between current VLA approaches and our SELF-VLA framework.

and generated. OpenVLA [12] discretizes robot actions into tokens and predicts single-step actions autoregressively using a large language model backbone. OpenVLA-OFT [13] replaces tokenized actions with a continuous regression head and introduces action chunking to predict short-term future sequences with fewer policy inference calls. $\pi_{0.5}$ [14] differs by employing a flow matching action head that generates continuous action chunks from noise. Octo [28] adopts a conditional diffusion policy [29] head to model continuous action chunks through multi-step denoising.

While VLA models integrate multi-modal perception and robot action in an end-to-end manner, their applicability in real-world applications remains challenging [30]. Liu et al. [18] conduct a case study evaluating different VLA models in contact-rich disassembly tasks. The evaluated models achieved near-zero task success even after fine-tuning, with reported difficulties including insufficient localization precision and unintended collisions. They also explored a hybrid strategy that switched from VLA control to position control at a specified height, with limited success. Lu et al. [31] propose VLA-RL, an online reinforcement learning framework that improves autoregressive OpenVLA policies through interaction beyond offline demonstrations.

Agentic frameworks can organize robotic task execution through model coordination and tool use. Yang et al. [32] propose a standardized action procedure (SAP) that coordinates an LLM-based planner, a VLA executor, and a vision-language model (VLM) verifier for closed-loop manipulation with failure recovery. Zhao et al. [33] propose an agentic extension of OpenVLA that integrates task planning, web search, object grounding, and memory-based retrieval to improve concept-level generalization in robotic manipulation.

At the policy level, MemoryVLA [34] is proposed as an end-to-end VLA model that integrates working memory and a perceptual-cognitive memory bank to provide temporal context for action generation. MEM [35] is a multi-scale embodied memory system that combines video-encoded short-term observations with long-term textual summaries to retain recent perceptual detail and track task progress in long-horizon tasks. Pang et al. [36] introduce AtomBridge, which retrieves pre-contact ready states from demonstrations and uses an LLM to generate transitions between VLA-executed atomic tasks. Graph-as-Policy (GaP) [37] organizes multi-agent robot programming around executable computation graphs that serve as robot policies. An orchestration agent coordinates skill agents to construct subgraphs from a modular library of perception, planning, and control functions.

III. METHODOLOGIES

A. SELF-VLA

In our proposed agentic framework, we integrate VLA policies with a library of programmed manipulation skills for contact-rich disassembly. Each skill is a reusable manipulation procedure specified by waypoint sequences, comprising a relative-motion disassembly part and an absolute-motion placement part. The VLA-planner generates actions toward the selected skill's entry pose, and the skill executes the disassembly procedure. If a failure is detected after pickup or during transport, the VLA-corrector generates regripping actions. Stop-token prediction (STP) coordinates the planner-to-skill handoff and the corrector's handoff to the retained placement sequence. Placement proceeds only if no failure is detected after the initial or corrective pickup. SELF-VLA automatically switches between the planner and corrector

models. Fig. 2 compares SELF-VLA with end-to-end VLA control, and Algorithm 1 summarizes the interaction between the components.

1) *VLA-planner*: Given a selected component category k and its language instruction l , the VLA-planner π_θ uses the base and wrist camera observations o_t and measured robot state x_t to approach an entry pose suitable for the selected skill’s contact-rich manipulation. It predicts an action chunk

$$A_t = \pi_\theta(o_t, x_t, l), \quad A_t \in \mathbb{R}^{H \times 7}, \quad (1)$$

where H is the prediction horizon, set to 10 actions for $\pi_{0.5}$ and 8 for OpenVLA-OFT. The execution horizon matches H for each backbone. Each action contains six motion coordinates and a gripper output. The motion coordinates are joint targets for $\pi_{0.5}$ and Cartesian increments for OpenVLA-OFT.

2) *Skill library*: Each component category k has a programmed skill $\mathcal{G}_k$ that combines its disassembly sequence with a predefined placement sequence according to the initial task instruction l . We write $(\tau_{\text{dis}}, \tau_{\text{plc}}) = \mathcal{G}_k(x_{\text{entry}}, l)$, where x_{entry} is the robot state at skill invocation. The disassembly part, τ_{dis} , performs disassembly through pickup using relative waypoints referenced to the initial tool center point (TCP) pose established by the planner. The placement part, τ_{plc} , uses absolute waypoints in the robot-base frame to place the component at the desired destination.

The library contains seven component-specific skills, with the same skill reused across variants within each category. After pickup or during transport, the failure detector compares the observed gripper width w_{obs} with the commanded closing width w_{cmd} using a predefined threshold ϵ_g . A detected failure initiates correction. Otherwise, the skill advances to τ_{plc} .

3) *Stop-token prediction*: We encode policy termination in the existing gripper output channel using $g_{\text{stop}} = 255$, which is reserved as a stop token and unused by normal gripper commands and observations in our task encoding. The planner and corrector learn this convention during fine-tuning without changing the VLA architecture or output dimensionality. The token provides termination evidence for control transfer and is not executed as a gripper command.

STP uses a temporal classifier for planner termination and direct recognition of the trained stop token for corrector termination. To reduce premature handoffs caused by isolated stop predictions, the temporal classifier evaluates each planner action chunk together with recent prediction and measured-state histories $\mathcal{H}_t$. We represent $\mathcal{H}_t$ with six time slots spanning the most recent 0.5 s.

From the current chunk, we extract a feature c_t containing gripper predictions and motion magnitudes. For $\pi_{0.5}$, these magnitudes are computed from predicted joint targets relative to measured joints. For OpenVLA-OFT, translation and rotation increment magnitudes are represented separately. A GRU encodes $\mathcal{H}_t$, and its final hidden state h_t is fused with c_t and the component embedding e_k :

$$\begin{aligned} h_t &= \text{GRU}_\omega(\mathcal{H}_t), \\ p_t &= \mathcal{C}(f_\omega([h_t, c_t, e_k])), \end{aligned} \quad (2)$$

Algorithm 1 SELF-VLA

Require: Component k , instruction l , planner π_θ , corrector π_ϕ , skill $\mathcal{G}_k$, STP

```

1: repeat
2:   Read observations  $o_t$  and robot state  $x_t$ 
3:    $A_t \leftarrow \pi_\theta(o_t, x_t, l)$ 
4:   Evaluate STP using  $A_t$ , recent history, and  $k$ 
5:   if planner termination is not confirmed then
6:     Execute the planner’s ordinary approaching actions
7:   end if
8: until planner termination is confirmed
9:  $(\tau_{\text{dis}}, \tau_{\text{plc}}) \leftarrow \mathcal{G}_k(x_t, l)$ 
10: Execute  $\tau_{\text{dis}}$  through pickup
11: Run failure detection:  $w_{\text{obs}} - w_{\text{cmd}} < \epsilon_g$ 
12: while a failure is detected do
13:   repeat
14:     Observe  $o_t, x_t$  and obtain corrector action  $a_t$  from  $\pi_\phi$ 
15:     if  $a_t$  does not encode the stop token then
16:       Execute the regrasping action  $a_t$ 
17:     end if
18:   until STP detects the stop token in the current corrector action  $a_t$ 
19:   Repeat failure detection using the same width criterion
20: end while
21: Execute  $\tau_{\text{plc}}$  if no failure is detected, invoking the correction block above upon a detected transport failure

```

where f_ω is a multilayer perceptron and $\mathcal{C}$ is the probability-calibration mapping. STP evaluates each newly generated planner action chunk once and declares STOP only when p_t exceeds the decision threshold and the chunk contains a stop token. The planner-to-skill handoff occurs after STOP is declared for three consecutive chunks.

We train the temporal classifier with approach-termination supervision while keeping the planner frozen. Each backbone uses its own classifier parameters and training-derived feature-normalization statistics.

4) *VLA-corrector*: When a failure is detected after pickup or during transport, the VLA-corrector π_ϕ uses the task instruction, current camera observations, and robot state to generate regrasping actions. STP directly recognizes the corrector’s trained stop token to terminate corrective manipulation. After termination, the failure detector checks the recovered grasp. If it still detects a failure, the corrector reactivates. Otherwise, the skill resumes the retained τ_{plc} .

B. Dataset Construction

To train the agentic framework and the end-to-end VLA baselines, we construct a real-world disassembly dataset for high-value component extraction from EoL desktops in two stages. In the first stage, we construct the skill library by teleoperating the robot through contact-rich manipulation operations in the disassembly sequence and recording waypoints for seven component-specific skills. Each skill defines the corresponding disassembly operation and subsequent placement at a predefined destination. In the second stage, we collect full component disassembly demonstrations combining teleoperated approach and correction with programmed skill execution. The dataset contains 910 demonstration episodes, with 100 each for HDD and SSD (one variant

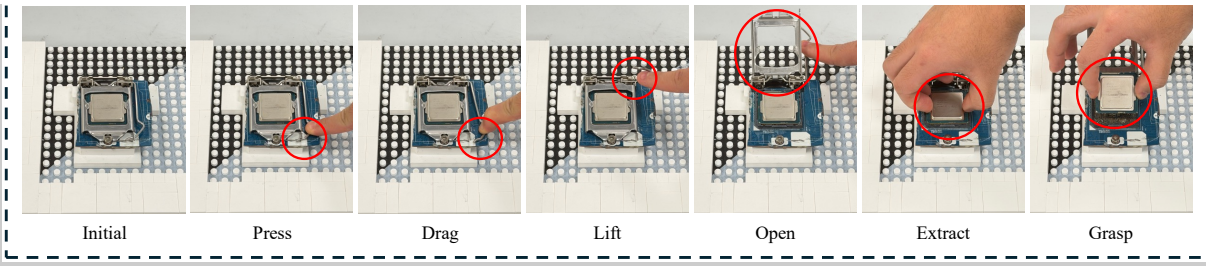


Fig. 3. The complexity of the CPU extraction task in EoL desktop disassembly.

each), 160 each for connector (two variants) and RAM (five variants), and 130 each for CPU fan (one variant), graphics card (five variants), and CPU (five variants). Within each category, the same skill is used for all component variants.

The target components are randomly positioned within a $45\text{cm} \times 45\text{cm}$ workspace, while their orientations remain fixed. All demonstrations are collected by a single operator using a dual-joystick interface. For each episode, the robot starts from a fixed home position, and the operator adjusts the end-effector position and orientation to reach a suitable starting pose for the corresponding skill. The operator then triggers skill execution. After the pickup action, the failure detector evaluates gripper feedback. If no failure is detected, the skill continues with placement. Otherwise, the operator provides corrective input through teleoperation to regrasp the component. After successful regrasping, the skill resumes only the placement procedure. The idle time between the robot’s pause after failure detection and the operator’s corrective input is excluded from the recording.

To enable separate training for the VLA-planner, VLA-corrector, and end-to-end baselines, demonstration episodes are annotated using four phase labels: approaching, skill execution, correction, and skill resumption. The approaching phase spans from the initial home position to the point where the skill is triggered. The skill execution phase covers the manipulation operation from skill invocation to placement completion or failure detection. When recovery is attempted, the correction phase captures the operator’s recovery input, followed by skill resumption if regrasping succeeds. The approaching-phase data are used to train the VLA-planner. To train the VLA-corrector for component regrasping, we supplement the operator-demonstrated correction trajectories with pickup-centered segments from the disassembly demonstrations. Each segment begins 2 s before the pickup action and ends after the subsequent failure detection step. These segments may span both the approaching and skill execution phases. Full episodes, including both teleoperated and skill-executed actions, are used to train the end-to-end VLA baselines.

We use LoRA to fine-tune OpenVLA-OFT [13] and $\pi_{0.5}$ [14] as planners, correctors, and end-to-end baselines. All policies use the original 30 Hz demonstrations, with the role-specific training data derived from the same episode collection as described above. Each policy is trained on all seven component categories and conditioned on the task instruction. All VLA policies are fine-tuned for 50,000 steps using the same training settings.

IV. EXPERIMENTS AND TEST RESULTS

A. Environmental Setup

We consider seven component disassembly tasks from EoL desktops: CPU fan, RAM, graphics card, connector, SSD, HDD, and CPU. These tasks require different combinations of approach accuracy and contact sequencing. CPU extraction involves unlocking the retention mechanism and grasping within the socket, as illustrated in Fig. 3. RAM removal requires alignment with a thin component in a constrained slot. CPU fan and connector removal also involve vertical operations with small and precise contact requirements. Graphics card removal requires two-stage disassembly, while HDD removal involves horizontal two-stage grasping of a relatively heavy component. SSD removal uses strict horizontal grasping. Each task ends with placement at a predefined destination.

The experimental platform consists of a UR5e robot arm equipped with a Robotiq 2F-85 gripper. A customized gripper extension provides access to contact locations needed for operations such as CPU bracket unlocking and extraction. The components are mounted in a tabletop workcell. For motherboard-mounted components, we use sections cut from actual motherboards to retain the component-level contact interfaces while reducing surrounding clutter. An Intel RealSense D435i mounted on the end effector provides the wrist view, and an OAK-D Pro mounted relative to the robot base provides the side view. The workcell, camera configuration, and robot-state acquisition setup are the same during demonstration collection and evaluation.

B. Results

We evaluate all methods in the same simplified workcell and board region used for demonstration collection to limit distribution shifts. Each component is tested in five configurations with 10 trials per configuration for each method. Repeated trials retain the same scene arrangement with small variations in component position. RAM and connector trials are balanced across the available slots. The configurations, task instructions, and robot home pose are matched across methods. Model switching occurs automatically upon a detected handoff. Success requires complete disassembly and placement at the designated destination. A trial is terminated and counted as a failure upon reaching the 150 s time limit or when the operator judges completion to be infeasible.

SELF-VLA outperforms the end-to-end (E2E) policies in all seven component categories, raising mean task success from 0.0% to 63.4% with OpenVLA-OFT and from 23.7% to

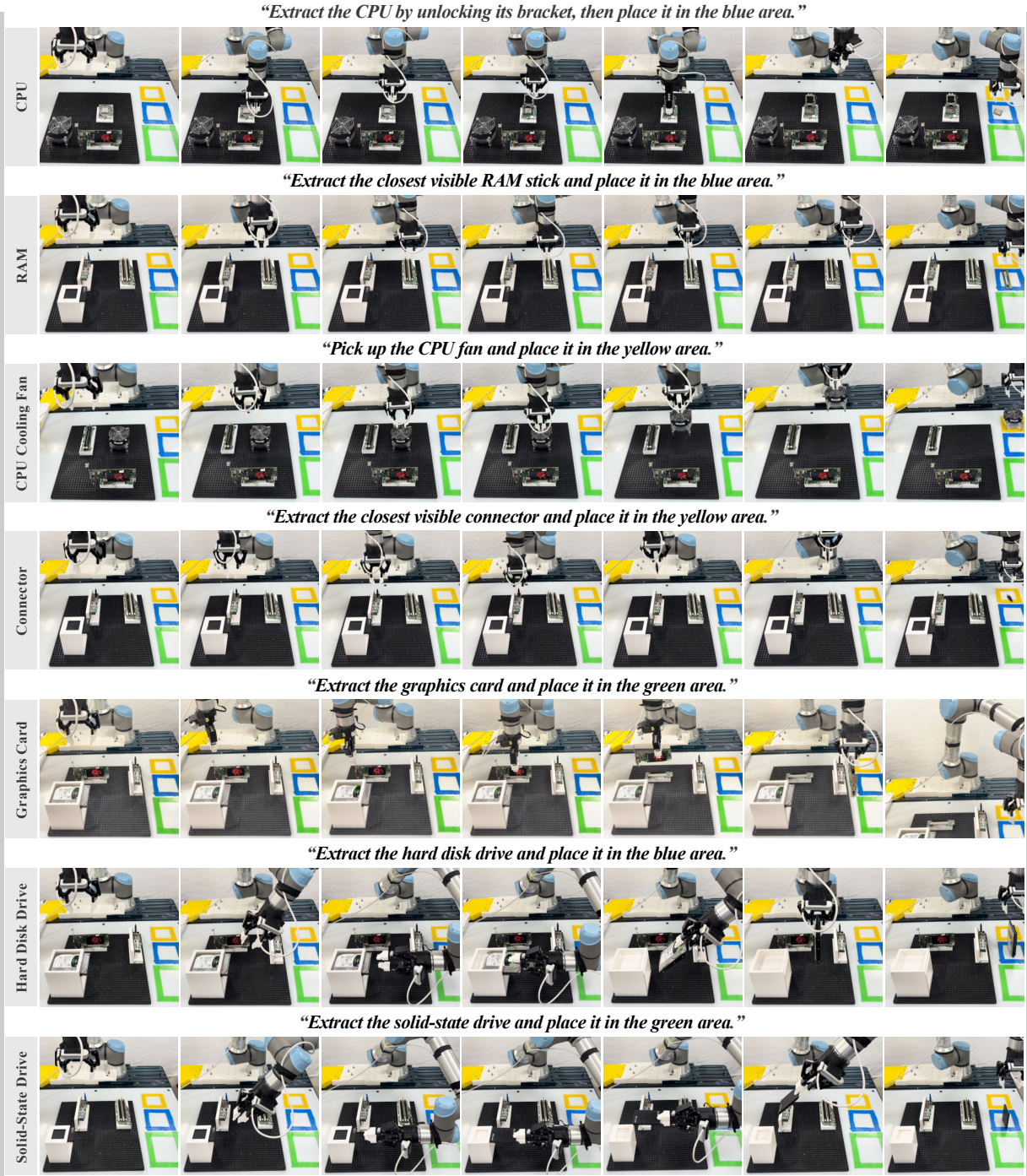


Fig. 4. Illustrative execution sequences for CPU, RAM, CPU cooling fan, connector, graphics card, hard disk drive (HDD), and solid-state drive (SSD). 76.0% with $\pi_{0.5}$ (Table I). The E2E policies demonstrate different grasping difficulties. OpenVLA-OFT frequently fails to close the gripper at the appropriate moment, while $\pi_{0.5}$ primarily struggles with inaccurate grasp positions. Fig. 4 shows representative execution sequences.

In SELF-VLA, the skill integration alleviates the impact of these difficulties by reducing reliance on VLA-generated contact actions and gripper commands during precision-sensitive manipulation. OFT-based SELF-VLA performs best on HDD and SSD. Their larger visual targets facilitate approach, while broader skill-entry regions accommodate positional variation. However, graphics card disassembly re-

quired a narrow local entry region and a two-stage operation despite the component’s large size. This contrast highlights local positioning and manipulation demands beyond overall size. Across different backbones, the $\pi_{0.5}$ framework achieves higher final task success in the other five categories.

To evaluate the effectiveness of the SELF-VLA framework, we use the $\pi_{0.5}$ backbone to compare STP, including its three-chunk confirmation gate, with the planner’s raw stop-token signal on an independent test set of 111 trajectories comprising 6,197 frames. A handoff signal is valid if it occurs within ± 0.1 s of the reference time at which the robot reaches a suitable skill-entry pose. The

TABLE I
FULL-TASK SUCCESS COUNTS FOR END-TO-END CONTROL AND SELF-VLA.

Backbone	Method	CPU fan	RAM	Graphics card	Connector	SSD	HDD	CPU
OpenVLA-OFT	End-to-End	0/50	0/50	0/50	0/50	0/50	0/50	0/50
	SELF-VLA	37/50	21/50	29/50	23/50	42/50	46/50	24/50
$\pi_{0.5}$	End-to-End	2/50	13/50	22/50	11/50	8/50	27/50	0/50
	DINOv2 trigger	14/50	12/50	18/50	12/50	16/50	20/50	16/50
	Successes without correction ^a	33/50	28/50	39/50	30/50	39/50	44/50	29/50
	SELF-VLA	40/50	34/50	39/50	32/50	39/50	44/50	38/50

Entries report full-task successes out of 50 trials per component. Bold counts indicate the higher full SELF-VLA result across backbones. DINOv2 trigger replaces only the planner-to-skill termination decision.

^a Successes without correction reports complete-task successes without correction within the same full $\pi_{0.5}$ SELF-VLA trials.

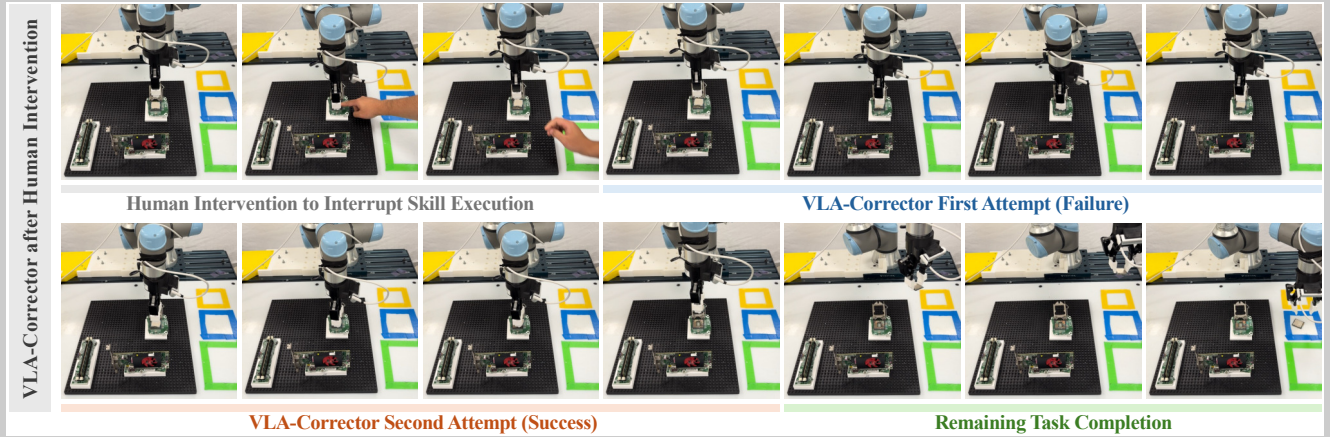


Fig. 5. Illustrative VLA-corrector execution after a human interrupts the skill execution. The corrector regrips the CPU, and the skill resumes placement.

valid-handoff rate is the proportion of all evaluated cases with a valid signal, and cases without a trigger are counted as invalid. STP achieves a valid-handoff rate of 84.68%, compared with 48.65% for the planner's raw stop-token signal. We further evaluate this handoff by replacing the STP-based trigger with a DINOv2-based trigger. We use the same training dataset as for STP training for the DINOv2 model and evaluate performance on hardware rollout tests using DINOv2 predictions directly. The test results show that the DINOv2 method reduced the task success from 76.0% to 30.9%. The trigger frequently terminates the approach prematurely, initiating contact-rich actions from unsuitable start poses and causing skill execution failures.

We also evaluate the contribution of the corrector in SELF-VLA. When the failure detector identifies a failure during skill execution, we invoke the corrector to attempt recovery. In the $\pi_{0.5}$ trials, correction improves task completion, particularly for CPU tasks, where pickup failures often occur after the component has been released from its bracket. A local regrasp can therefore complete the pickup. Fig. 5 shows a recovery sequence following a human interruption of the CPU disassembly skill execution. Other successful recoveries typically follow initial pickup failures that leave components near their original mounting positions. In the observed HDD, SSD, and graphics card trials, skill-execution failures are caused by unsuitable entry poses or component drops during

transport. The OpenVLA-OFT corrector continues to struggle with gripper closure timing and adds no task completions. These observations suggest that the planner's approach and STP-guided handoff support successful skill execution from suitable entry poses.

V. CONCLUSIONS

We present SELF-VLA, a skill-enhanced agentic framework for contact-rich disassembly. It combines a planner for approaching the target component, a skill library for disassembly and placement, and a corrector for recovery. A learned stop-token prediction (STP) module determines when the planner transfers control to a skill, while the failure detector triggers recovery upon detecting pickup failures. This coordination substantially improves task success compared with relying entirely on a VLA policy throughout task execution. However, out-of-distribution states remain a major challenge, particularly when the planner fails to reach suitable skill-entry poses. These failures reflect the framework's dependence on the control accuracy and generalization capability of its VLA backbone. Future work could explore replacing STP with a vision-language model (VLM) trained to assess handoff readiness across a broader range of situations. Such a model could also serve as the orchestration core of the agentic VLA frameworks, coordinating the planner, skill execution, and recovery.

ACKNOWLEDGMENTS

Codex and Claude were used to assist with experimental code generation and implementation. The authors reviewed the generated code.

REFERENCES

- [1] T. Maani, S. Deng, and J. W. Sutherland, "Disassembly analysis to promote rare earth permanent magnet recovery from end-of-life electric vehicle motors," in *International Manufacturing Science and Engineering Conference*, vol. 88100. American Society of Mechanical Engineers, 2024, p. V001T04A004.
- [2] M.-L. Lee, X. Liang, B. Hu, G. Onel, S. Behdad, and M. Zheng, "A review of prospects and opportunities in disassembly with human-robot collaboration," *Journal of Manufacturing Science and Engineering*, vol. 146, no. 2, p. 020902, 2024.
- [3] C. Liu, B. Balasubramaniam, N. A. Yancey, M. H. Severson, A. Shine, P. Bove, B. Li, X. Liang, and M. Zheng, "Raise: A robot-assisted selective disassembly and sorting system for end-of-life phones," *Resources, Conservation and Recycling*, vol. 225, p. 108609, 2026.
- [4] L. Andeobu, S. Wibowo, and S. Grandhi, "Informal e-waste recycling practices and environmental pollution in africa: What is the way forward?" *International journal of hygiene and environmental health*, vol. 252, p. 114192, 2023.
- [5] W. Deng, Q. Liu, Y. Wang, F. Zhao, Y. Cai, Z. Zhou, and D. T. Pham, "Learning from new products: A robust end-of-life object detection model for robotic disassembly using the dual constraints of anchors and corners," *Journal of Cleaner Production*, vol. 518, p. 145882, 2025.
- [6] J. Yu, H. Zhang, Z. Jiang, W. Yan, Y. Wang, and Q. Zhou, "Disassembly task planning for end-of-life automotive traction batteries based on ontology and partial destructive rules," *Journal of Manufacturing Systems*, vol. 62, pp. 347–366, 2022.
- [7] M.-L. Lee, S. Behdad, X. Liang, and M. Zheng, "Task allocation and planning for product disassembly with human-robot collaboration," *Robotics and Computer-Integrated Manufacturing*, vol. 76, p. 102306, 2022.
- [8] M. E. Asif, A. Rastegarpanah, and R. Stolkin, "Robotic disassembly for end-of-life products focusing on task and motion planning: A comprehensive survey," *Journal of Manufacturing Systems*, vol. 77, pp. 483–524, 2024.
- [9] D. Sliwowski, S. Javad, S. Stanovcic, J. Orbik, J. Heidersberger, and D. Lee, "Reassemble: A multimodal dataset for contact-rich robotic assembly and disassembly," *arXiv preprint arXiv:2502.05086*, 2025.
- [10] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 5, pp. 1–72, 2025.
- [11] K. Kawaharazuka, J. Oh, J. Yamada, I. Posner, and Y. Zhu, "Vision-language-action models for robotics: A review towards real-world applications," *IEEE Access*, 2025.
- [12] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, "Open-vla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.
- [13] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025.
- [14] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, " $\pi_{0.5}$: a vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.
- [15] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [16] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.
- [17] S. Li, Y. Chen, D. Li, S. Liu, D. Lan, Y. Liu, and Z. Pang, "Transferring vision-language-action models to industry applications: Architectures, performance, and challenges," in *2025 7th International Conference on Industrial Artificial Intelligence (IAI)*. IEEE, 2025, pp. 1–6.
- [18] C. Liu, S. Tian, S. Behdad, X. Liang, and M. Zheng, "Vision-language-action models for selective robotic disassembly: A case study on critical component extraction from desktops," *arXiv preprint arXiv:2512.04446*, 2025.
- [19] P. Guo, J. Wang, C. Zheng, and W. Qiang, "Papo-vla: Planning-aware policy optimization for vision-language-action models," *arXiv preprint arXiv:2605.19580*, 2026.
- [20] D. B. Acharya, K. Kuppan, and B. Divya, "Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey," *IEEE Access*, vol. 13, pp. 18 912–18 936, 2025.
- [21] R. Xu and Y. Yan, "Agent skills for large language models: Architecture, acquisition, security, and the path forward," *arXiv preprint arXiv:2602.12430*, 2026.
- [22] J. P. J. Prioli, H. M. Alrufaifi, and J. L. Rickli, "Disassembly assessment from cad-based collision evaluation for sequence planning," *Robotics and Computer-Integrated Manufacturing*, vol. 78, p. 102416, 2022.
- [23] X. Zhang, K. Eltouny, X. Liang, and S. Behdad, "Automatic screw detection and tool recommendation system for robotic disassembly," *Journal of Manufacturing Science and Engineering*, vol. 145, no. 3, p. 031008, 2023.
- [24] G. Foo, S. Kara, and M. Pagnucco, "Challenges of robotic disassembly in practice," *Procedia CIRP*, vol. 105, pp. 513–518, 2022.
- [25] S. Hjorth and D. Chrysostomou, "Human-robot collaboration in industrial environments: A literature review on non-destructive disassembly," *Robotics and Computer-Integrated Manufacturing*, vol. 73, p. 102208, 2022.
- [26] S. Tian, M. Zheng, and X. Liang, "Real-time 3d motion prediction for human-robot collaboration via bayesian-optimized diffusion models," *IEEE Transactions on Automation Science and Engineering*, 2025.
- [27] S. K. Hopko, R. K. Mehta, and P. R. Pagilla, "Physiological and perceptual consequences of trust in collaborative robots: An empirical investigation of human and robot factors," *Applied Ergonomics*, vol. 106, p. 103863, 2023.
- [28] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.
- [29] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
- [30] M. U. Din, W. Akram, L. S. Saoud, J. Rosell, and I. Hussain, "Vision language action models in robotic manipulation: A systematic review," *arXiv preprint arXiv:2507.10672*, 2025.
- [31] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang, "Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning," *arXiv preprint arXiv:2505.18719*, 2025.
- [32] Z. Yang, Y. Chen, X. Zhou, J. Yan, D. Song, Y. Liu, Y. Li, Y. Zhang, P. Zhou, H. Chen, *et al.*, "Agentic robot: A brain-inspired framework for vision-language-action models in embodied agents," *arXiv preprint arXiv:2505.23450*, 2025.
- [33] H. Zhao, J. Zhang, W. Song, P. Ding, and D. Wang, "Vla²: Empowering vision-language-action models with an agentic framework for unseen concept manipulation," *arXiv preprint arXiv:2510.14902*, 2025.
- [34] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang, "Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation," *arXiv preprint arXiv:2508.19236*, 2025.
- [35] M. Torme, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, *et al.*, "Mem: Multi-scale embodied memory for vision language action models," *arXiv preprint arXiv:2603.03596*, 2026.
- [36] Y. Pang, B. Zhou, C. Li, X. Wang, S. Xu, D.-B. Wang, M.-L. Zhang, and S. Di, "Sci-vla: Agentic vla inference plugin for long-horizon tasks in scientific experiments," *arXiv preprint arXiv:2602.09430*, 2026.
- [37] K. Chen, S. Xie, L. Fu, J. Yu, W. Pacini, S. Bajamahal, H. Kim, J. Drake, D. Kim, H. Xue, J. Francis, C. Juette, P. Schaldenbrand, M. Y. Seker, R. Wickramarachchi, U. Yoo, G. Wang, A. Murali, B. Sundaralingam, S. S. Sastry, S. Huang, Y. Zhu, L. Fan, and K. Goldberg, "Gap: A graph-as-policy multi-agent self-learning harness for variational automation tasks," *arXiv preprint arXiv:2607.05369*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.05369>